ATLAS

Desenvolupament d'un agent d'intel·ligència artificial conversacional amb capacitat d'actuació integrat en un prototip físic accessible

Sami González Kamel
Grup: 2nBATA
Centre: IESTARRAGONA
Tutor: Rafael Roca Campos

21 d'octubre de 2026
Memòria final

Resumen

En este trabajo investigué cómo construir un agente conversacional capaz de actuar sobre su entorno e integrarlo en un dispositivo físico accesible. El objetivo era ir más allá de un chat: quería que ATLAS interpretara peticiones, utilizara herramientas y conservara información relevante con suficiente rapidez para mantener una conversación.

Seguí un proceso experimental basado en prototipos, documentación técnica, pruebas y revisión de errores. Inicialmente utilicé OpenClaw para conectar el modelo, la memoria y las herramientas. Esta estructura validó la idea, pero la cadena de transcripción, respuesta y síntesis de voz añadía demasiada espera. Por eso desarrollé una arquitectura propia centrada en GPT Realtime y herramientas directas.

El prototipo integra una Raspberry Pi 5, una pantalla táctil, un micrófono y altavoces. Dispone de WebScreen, chat de terminal, aplicación Android, integración con WhatsApp y un portal web protegido. Incorpora rutinas y un catálogo de 323 expresiones. La carcasa quedó completada como un modelo tridimensional digital.

Las pruebas comprueban la viabilidad del conjunto y muestran la diferencia entre recibir el primer fragmento de respuesta y obtener un resultado útil. Registré recorridos de voz, texto y control de Android, comparé versiones con 400 peticiones y preparé una segunda batería de 400 interacciones nuevas. Las acciones potencialmente molestas se aislaron en un entorno virtual.

La conclusión principal es que la experiencia depende tanto de la coordinación del sistema como del modelo escogido. ATLAS es un prototipo funcional que combina conversación, memoria, herramientas y hardware propio, aunque continúa dependiendo de servicios externos para la conversación.

Abstract

In this project, I investigated how to build a conversational agent that can act on its environment and run through an accessible physical device. My aim was to go beyond a chat interface: ATLAS should interpret requests, use tools and retain relevant information quickly enough to support a conversation.

I followed an experimental process based on prototypes, technical documentation, testing and error analysis. I initially used OpenClaw to connect the model, memory and tools. This validated the idea, but transcription, response and speech synthesis introduced too much delay. I therefore developed an independent architecture centred on GPT Realtime and direct tool execution.

The prototype combines a Raspberry Pi 5, touchscreen, microphone and speakers. It provides WebScreen, a terminal chat, an Android application, WhatsApp integration and a protected web portal. It also includes routines for repetitive tasks and a catalogue of 323 expressions. The enclosure was completed as a digital three-dimensional model.

The tests support the feasibility of the system and show the difference between receiving the first response fragment and obtaining a useful result. I recorded voice, text and Android control flows, compared versions with 400 requests and prepared a second battery of 400 new interactions. Potentially disruptive actions were isolated in a virtual environment.

The main conclusion is that coordination between components matters as much as model capability. ATLAS is a working prototype that combines conversation, memory, tools and dedicated hardware, although it still depends on external services for its main conversational functions.

Agraïments

Vull agrair el suport de les persones que m'han acompanyat durant aquest treball.

Al meu pare, pel seu suport moral, per la supervisió de la memòria escrita i per la seva ajuda general durant el projecte.

A l'institut, per brindar-me la possibilitat d'utilitzar la impressora 3D per al projecte.

A Pedro Jesús Ortega Papaseit, per la seva ajuda amb la impressora 3D i per la correcció de la memòria escrita.

A Alex Viejo, coordinador del Grau d'Enginyeria Informàtica de la URV, per revisar el prototip i la proposta inicial del projecte, i pels seus consells i idees, que han sigut molt valuosos pel desenvolupament de la part pràctica i teòrica.

I a Rafael Roca Campos, el meu tutor de TDR, pels consells i les idees que m'ha aportat al llarg d'aquest.

Índex

Resumen **2**

Abstract **3**

Agraïments **4**

Introducció **7**

Objectius **8**

Antecedents **9**

I. INFORMÀTICA — L'agent i el seu programari **10**

1. Concepte de l'agent conversacional **10**

1.1. Definició d'agent 10

1.2. El nom i la identitat d'ATLAS 10

1.3. Desenvolupament propi i components reutilitzats 11

2. Evolució d'OpenClaw a Realtime **12**

2.1. Base del prototip inicial 12

2.2. El flux de veu inicial 13

2.3. Transcripció en temps real i preàmbuls 13

2.4. Motius del canvi d'arquitectura 14

2.5. Components conservats i retirats d'OpenClaw 15

3. Models, context i eines **15**

3.1. Concepte de token 15

3.2. Context i límits de memòria 16

3.3. Memòria entre sessions 17

3.4. Eines i resultats verificables 17

4. Arquitectura pròpia d'ATLAS **18**

4.1. Components i distribució de funcions 18

4.2. Native Broker i preparació de les sessions 18

4.3. Organització d'arxius i context 19

4.4. Rutines per a tasques repetitives 20

4.5. Perfils i límits de capacitats 20

5. Interfícies i connexió amb altres dispositius **21**

5.1. WebScreen i conversa per veu 21

5.2. Veu nativa i motors alternatius 22

5.3. El xat de terminal com a eina de proves 23

5.4. Aplicació Android i servei Companion 23

5.5. Aparellament i transport privat 24

5.6. Control natiu i control visual del telèfon 25

5.7. Integració de WhatsApp 26

5.8. Accés des de la web 26

5.9. Expressivitat de la interfície 27

6. Fiabilitat, seguretat i manteniment **28**

6.1. Control de torns i recuperació de connexions 28

6.2. Soroll, interrupcions i naturalitat 28

6.3. Dades privades i permisos 29

6.4. Diagnòstic i manteniment 30

II. ELECTRÒNICA — El dispositiu ATLAS A1 **31**
7. Selecció de components i integració física **31**
7.1. Selecció de la Raspberry Pi 31
7.2. Pantalla i entrada tàctil 32
7.3. Micròfon, altaveus i Bluetooth 33
7.4. Alimentació, emmagatzematge i refrigeració 34
8. ATLAS OS, recuperació i disseny de la carcassa **34**
8.1. Adaptació del sistema operatiu 34
8.2. Modes d'ús i recuperació local 35
8.3. Disseny digital de la carcassa 36
8.4. Configuració per a Bambu Lab H2S 38
8.5. Pressupost del prototip 39
8.6. Ampliacions físiques i criteri de selecció 39
III. AVALUACIÓ — Resultats i límits **40**
9. Avaluació del prototip **40**
9.1. Mesurament de l'experiència completa 40
9.2. Resultats dels mesuraments històrics 40
9.3. Proves de veu, rutines i Android 41
9.4. Bateries de preguntes i millora de les accions 42
9.5. Optimització de la rapidesa i la qualitat 43
9.6. Límits de l'avaluació 43
IV. CONCLUSIONS I CONTINUÏTAT DEL PROJECTE **45**
10. Resposta a la pregunta inicial **45**
11. Aprenentatges de la integració del sistema **45**
12. Desenvolupament futur **46**
Referències documentals **48**

Introducció

Des del primer moment, la meva idea principal va ser crear una intel·ligència artificial capaç de fer alguna cosa més que respondre preguntes. Volia parlar amb ella de manera natural, que recordés informació important i que, quan li demanés una tasca, tingués alguna manera de dur-la a terme. No m'interessava únicament construir un xat; volia convertir-lo en un agent amb un entorn en el qual actuar.

Els assistents de veu van ser una referència. M'agradava demanar alguna cosa sense obrir una aplicació, però volia experimentar amb un sistema que pogués adaptar a les meves necessitats. A partir d'aquí vaig formular la pregunta principal: era possible construir un agent conversacional, amb memòria i eines, i integrar-lo en un dispositiu físic accessible, mantenint una interacció prou ràpida?

La meva hipòtesi va ser que sí que era possible, combinant un model de llenguatge amb programari propi i una Raspberry Pi. La recerca es va centrar en connectar aquestes peces, resoldre les seves limitacions i comprovar fins a quin punt el resultat podia actuar de manera útil.

Per a comprovar-ho em vaig proposar construir un flux de veu funcional, proporcionar context persistent i eines, crear interfícies per a la pantalla, la terminal i Android, i integrar micròfon i altaveus. També havia d'avaluar la rapidesa i la fiabilitat de les accions, i permetre que una persona pogués diagnosticar el dispositiu quan alguna cosa fallés.

La metodologia va ser de desenvolupament experimental: vaig preparar una primera solució, vaig observar els seus problemes, vaig registrar proves i vaig anar modificant l'arquitectura. Vaig utilitzar documentació tècnica per a entendre els components i vaig contrastar les seves especificacions amb el prototip. Alguns assajos van comprovar funcions de programari; uns altres van incloure dispositius reals. En l'avaluació distingeixo tots dos casos.

Vaig organitzar la memòria en quatre parts: informàtica, electrònica, avaluació i conclusions. Els catàlegs i registres detallats van quedar en els annexos. La memòria recull el desenvolupament del prototip i els seus resultats; l'apartat final reuneix les propostes de continuïtat. La meva intenció va ser explicar un procés real, amb decisions encertades i d'altres que vaig haver de canviar.

Objectius

El meu objectiu principal era crear un dispositiu físic que executés un agent d'intel·ligència artificial, amb el seu propi entorn de programari i eines. Volia poder parlar amb ell de manera natural i que fos capaç de fer alguna cosa més que respondre preguntes: recordar informació útil, entendre el que li demanava i actuar sobre el seu entorn quan fos necessari.

Per a aconseguir-ho, em vaig plantejar integrar una Raspberry Pi amb una pantalla, un micròfon i altaveus. La idea era reunir aquests components en un prototip que pogués utilitzar i modificar, sense dependre d'un ordinador d'escriptori per a cada interacció. També volia dissenyar una carcassa que permetés organitzar les peces, accedir als connectors i mantenir una ventilació adequada.

Un altre objectiu era preparar un sistema operatiu adaptat al dispositiu. No pretenia programar un des de zero, sinó partir de Linux i configurar un entorn propi per a ATLAS, amb els serveis, les eines i la interfície que necessités. De la mateixa manera, utilitzaria un model d'intel·ligència artificial existent: el meu treball se centrava en integrar-lo i donar-li una manera d'interactuar amb el sistema.

En la conversa, buscava que l'agent mantingués el fil entre diverses preguntes i conservés la informació important. També volia afegir eines per a consultar l'estat del dispositiu i fer tasques, de manera que no hagués d'indicar sempre el nom d'una ordre per a aconseguir alguna cosa.

Finalment, em vaig proposar comprovar la rapidesa, la fiabilitat i la utilitat del prototip mitjançant proves. Volia registrar els resultats, revisar els errors i comparar els canvis que anés fent, de manera que les millores es recolzessin en dades.

Antecedents

Quan vaig començar a plantejar ATLAS, la idea de parlar amb un dispositiu físic ja estava present en moltes llars. Els altaveus i pantalles intel·ligents eren una referència per al projecte: permeten fer preguntes, escoltar música o controlar dispositius compatibles sense utilitzar contínuament un teclat o un mòbil. Volia partir d'aquest concepte, però construir un prototip el funcionament del qual pogués desenvolupar i modificar jo mateix.

Entre els exemples més coneguts està la família Amazon Echo. Els Echo i Echo Dot integren Alexa en un altaveu, mentre que els Echo Show incorporen una pantalla per a complementar la interacció per veu. Aquesta permet consultar informació visual, reproduir contingut i utilitzar funcions com les videotrucades. Per a ATLAS, el format amb pantalla era especialment interessant, perquè volia que la interacció no depengués únicament del so (Amazon, 2020).

Google també té dispositius d'aquest tipus. Google Home i els posteriors models Nest combinen altaveus, micròfons i un assistent de veu. Nest Mini manté un format petit centrat en l'àudio, mentre que Nest Hub afegeix una pantalla per a mostrar informació, fotografies i controls de la llar. Són dues maneres d'integrar un assistent en un objecte quotidià: una més discreta i una altra amb una interfície visual (Google, 2019a i 2019b).

Un altre antecedent és el HomePod mini d'Apple, que incorpora Siri i permet escoltar música, realitzar consultes i controlar accessoris domèstics compatibles. També destaca la seva integració amb altres dispositius i serveis del mateix ecosistema (Apple, 2020).

ATLAS comparteix amb aquests productes la idea de tenir un assistent accessible mitjançant la veu i connectat a eines. La diferència que buscava era poder decidir com s'organitza el meu propi sistema: treballar sobre una Raspberry Pi, preparar l'entorn basat en Debian, dissenyar la interfície i afegir les eines que necessités. No pretenia superar tots aquests dispositius ni crear un model d'intel·ligència artificial des de zero, sinó comprovar fins a on podia arribar integrant models existents en un dispositiu propi.

I. INFORMÀTICA — L'agent i el seu programari

1. Concepte de l'agent conversacional

1.1. Definició d'agent

Una conversa per si sola no permet actuar sobre el món. Un model pot explicar com obrir una aplicació, però això no significa que pugui obrir-la. Perquè ho faci necessita una eina: una funció externa que rebi una petició, executi l'operació i retorni el resultat. Aquesta diferència va ser la base del projecte.

En ATLAS vaig utilitzar el terme agent per a referir-me al conjunt format pel model, les seves instruccions, el context, les eines i el programa que coordina tot. El model interpreta la petició i proposa què fer; el programari comprova i executa les operacions permeses; després, el resultat torna a la conversa. Si una eina falla, l'agent ha de rebre aquesta fallada, no assumir que la tasca es va completar.

Per exemple, davant "obre Amazon i cerca ESP32", respondre "ja està" no demostra res. Obrir l'aplicació tampoc completa la petició. El sistema necessita continuar fins a introduir la cerca i comprovar que apareix el resultat esperat. Aquesta distinció entre intenció, acció i verificació va resultar molt més important del que semblava en començar.

L'autonomia que buscava no era il·limitada. Consistia en el fet que ATLAS pogués triar passos intermedis sense obligar-me a descriure cada pulsació, sempre dins dels permisos concedits. No significa que qualsevol text que vegi hagi de convertir-se en una ordre, ni que hagi de realitzar accions irreversibles sense tenir clara la intenció de l'usuari.

1.2. El nom i la identitat d'ATLAS

Una vegada pensada la base del projecte, tocava donar-li un nom. Volia que fos curt, fàcil de pronunciar i còmode de repetir. També m'agradava que tingués algun significat relacionat amb la seva funció. Finalment vaig triar ATLAS: un nom que vaig associar tant a la idea de sostenir com a una col·lecció organitzada d'informació.

Durant la primera etapa vaig utilitzar també el nom OpenAtlas, per la relació amb OpenClaw. Més endavant, en construir una arquitectura independent, el nom va quedar simplement en ATLAS. ATLAS A1 identifica el dispositiu físic; ATLAS OS,

l'adaptació del sistema operatiu; i WebScreen, la interfície que apareix en la seva pantalla.

Aquesta separació evita confondre l'agent amb una sola aplicació. Si parlo amb ell des del mòbil o des de la terminal, continua sent ATLAS. El que canvia és el canal d'entrada i sortida, no la identitat ni el coneixement principal que rep.

Vaig buscar una personalitat pròxima, amb respostes breus i clares. La cara de la pantalla i les seves expressions acompanyen aquesta intenció, però no representen sentiments reals. Són recursos de comunicació que ajuden a reconèixer quan escolta, respon o està esperant. El valor principal del projecte continua estant a comprendre la petició i actuar correctament.



Figura 1. Símbol i nom d'ATLAS sobre una composició gràfica per a la web. Identitat pròpia; fons elaborat amb suport de generació d'imatges.

1.3. Desenvolupament propi i components reutilitzats

Per a construir el prototip vaig aprofitar tecnologies existents. El model d'intel·ligència artificial, Linux, el navegador i les biblioteques de comunicació no són desenvolupaments meus. El meu treball va consistir a integrar-los, definir les seves responsabilitats i crear les interfícies, eines i mecanismes de coordinació específics d'ATLAS.

Això no va eliminar la dificultat tècnica. Una funció podia funcionar aïllada i fallar en combinar-la amb una altra: el micròfon podia obrir-se correctament i, així i tot, entrar en conflicte amb una segona sessió; una aplicació podia mantenir connexió amb la Raspberry Pi mentre la conversa de veu encara no estava preparada. Gran part del desenvolupament va ser resoldre aquestes relacions.

Vaig utilitzar Codex com a suport per a programar, revisar errors, preparar proves i organitzar documentació. L'ajuda d'una eina de desenvolupament no substitueix la necessitat d'entendre el que s'integra. Per això vaig procurar contrastar les seves propostes amb els registres i amb el comportament del dispositiu, i comprovar què demostrava cada prova.

Tampoc vaig crear un model que aprengués modificant els seus paràmetres i pesos cada vegada que parlàvem. L'adaptació progressiva del prototip es va fer mitjançant instruccions i memòria en arxius. És una manera de conservar informació entre sessions, però tècnicament no és un reentrenament. Aquesta precisió és important per a entendre què pot i què no pot fer el sistema. Entrenar un model d'aquesta grandària des de zero requeriria recursos molt superiors als disponibles per al projecte.

2. Evolució d'OpenClaw a Realtime

2.1. Base del prototip inicial

Inicialment, per a la proposta del prototip d'ATLAS, vaig utilitzar OpenClaw, un projecte per al desplegament i personalització d'agents d'intel·ligència artificial. La proposta era bona, fàcil de configurar i molt personalitzable. Em va permetre unir un model, instruccions, memòria i eines sense haver de desenvolupar tota aquesta estructura abans de comprovar si la idea tenia sentit.

Durant bastant temps va ser una part important del projecte. No el vaig descartar perquè no era capaç de fer tasques o perquè les seves respostes eren necessàriament dolentes. Al contrari: em va ajudar a construir el primer agent funcional i a descobrir què necessitava realment. La seva documentació va servir com a referència per a organitzar aquella primera etapa. [OpenClaw, documentació general]

El problema va aparèixer quan vaig comparar aquest funcionament amb l'experiència que volia aconseguir. Hi havia funcions i canals instal·lats que no utilitzava, i la conversa passava per massa etapes abans d'escoltar-se una resposta. La flexibilitat que resultava còmoda per a un prototip no es traduïa automàticament en una interacció ràpida.

El meu objectiu era un agent conversacional amb capacitat d'actuar, no un sistema que acceptés una ordre i romangués diversos segons en silenci. Aquesta diferència va acabar orientant l'evolució d'ATLAS més que qualsevol característica aïllada del model.

2.2. El flux de veu inicial

La primera arquitectura seguia una cadena bastant habitual. L'usuari parlava, el micròfon captava el so i el programa detectava quan acabava la intervenció. Després, un model local de transcripció, Whisper, convertia l'àudio en text. Aquest text s'enviava a l'agent gestionat per OpenClaw i, quan arribava la seva resposta, un motor de veu la pronunciava.

Cada component tenia una funció raonable, però depenia de l'anterior. Si la transcripció no havia acabat, l'agent no podia començar; si la resposta no estava disponible, el sintetitzador tampoc tenia res a reproduir. A més, la detecció del silenci afegia una espera que no es veia en el contingut de la conversa.

En l'ús d'aquella versió vaig observar esperes habituals d'entre 7 i 15 segons des del final de la meva intervenció fins a l'inici de la resposta parlada. Aquest interval descriu la meva experiència amb el prototip i la seva configuració, no una mesura universal d'OpenClaw ni un límit de tots els models que podia utilitzar. Vaig provar diferents models de proveïdors, però tots arribaven al mateix coll d'ampolla, per culpa de l'estructura d'OpenClaw a l'hora de les inferències.

Aquest espai de silenci es feia enorme. A vegades semblava que ATLAS no havia escoltat o que el sistema estava bloquejat. Vaig aprendre que la qualitat d'una conversa no depèn només que la resposta final sigui correcta: també importa el que ocorre mentre l'usuari l'espera.

2.3. Transcripció en temps real i preàmbuls

El primer canvi important va ser substituir, en l'entrada habitual, la transcripció local pel reconeixement que oferia Chrome. D'aquesta manera podia rebre text mentre parlava, sense esperar sempre a processar un enregistrament complet en acabar. Això va reduir una part del recorregut, encara que no va convertir el reconeixement del navegador en una funció necessàriament local o offline. [Web Speech API]

També vaig ajustar el criteri per a donar per acabada una frase. Un silenci massa llarg retardava tot; un massa curt tallava una intervenció normal en diverses parts. No era suficient baixar un número fins que semblés ràpid: calia comprovar què ocorria amb pauses, soroll i frases que continuaven després d'una petita interrupció.

La segona ajuda va ser el que vaig denominar "preàmbul". Vaig preparar una ruta més lleugera que analitzava la petició i deia una cosa breu mentre l'agent principal

continuava treballant. En certes proves vaig poder reduir l'espera total cap als 7 o 8 segons i fer que hi hagués una primera intervenció al voltant dels 3 segons. Aquesta primera frase no era encara el resultat de la tasca.

La idea va funcionar durant un temps, però va introduir un altre problema. Ara havia de coordinar dues respostes: una que anticipava el que s'anava a fer i una altra que coneixia el resultat real. A vegades el preàmbul no s'activava; unes altres, li faltava context o deia alguna cosa que no encaixava del tot. Havia reduït el silenci percebut, però no havia eliminat el treball que continuava executant-se.

2.4. Motius del canvi d'arquitectura

Em va costar bastant deixar de costat aquest plantejament. Havia invertit temps a fer-lo funcionar i cada millora semblava acostar-lo una mica més al resultat que buscava. No obstant això, conforme afegia capacitats, també creixien els estats que calia coordinar i els casos en els quals la conversa se sentia inconsistent.

En les configuracions que havia provat, fins i tot les respostes relativament senzilles podien trigar cinc o sis segons abans d'afegir les altres etapes. No vaig concloure que cap model pogués respondre més ràpid; vaig concloure que la meva estructura no estava resolent bé el problema. Continuar afinant una peça aïllada tenia cada vegada menys efecte sobre el conjunt.

L'alternativa va ser utilitzar un model preparat per a conversar en temps real: GPT Realtime. Aquest podia treballar amb àudio i text i sol·licitar eines dins de la mateixa sessió. Això em permetia canviar l'estructura del problema: en lloc de connectar transcriptor, agent i veu com tres blocs obligatòriament consecutius, podia mantenir una conversa preparada i enviar-li la intervenció directament. [OpenAI, converses Realtime]

En torns senzills i amb la sessió ja oberta, vaig començar a obtenir respostes de l'ordre d'un a tres segons. El canvi es notava molt més que un ajust visual o una frase d'espera. Així i tot, les tasques que necessitaven diverses eines continuaven depenent de la seva execució. Realtime va reduir una part fonamental de l'espera; no va fer instantani tot el que ATLAS podia fer.

2.5. Components conservats i retirats d'OpenClaw

La migració no va consistir simplement a desinstal·lar OpenClaw. Abans vaig haver de separar les dades que pertanyien al projecte dels serveis que els estaven utilitzant.

Vaig conservar els arxius d'identitat, memòria i instruccions, vaig revisar les seves relacions i els vaig traslladar a una estructura pròpia dins de `.atlas`.

També va ser necessari migrar les configuracions privades que continuaven sent útils, com les claus de serveis de veu o cerca, sense incloure-les en el repositori. L'autenticació es va mantenir separada del coneixement de l'agent. Una clau permet accedir a un servei; no és una informació que hagi de formar part del text que rep el model.

El sistema actual ja no necessita el Gateway ni els canals d'OpenClaw per a conversar. Les seves responsabilitats es van repartir entre el servidor de WebScreen, el component d'autenticació, el motor d'eines i la memòria pròpia. Mantenir aquesta distinció evita descriure l'arquitectura actual com si continués sent una configuració d'OpenClaw.

No obstant això, aquesta etapa continua sent rellevant per a la memòria. Va ser la base amb la qual vaig validar la idea i el punt de comparació que va justificar el canvi. Retirar-lo del producte no significa esborrar-lo de la història del projecte: precisament el procés de detectar els seus límits i substituir-lo forma part del treball de recerca.

3. Models, context i eines

3.1. Concepte de token

Arribat aquest punt, considero oportú explicar alguns conceptes que utilitzo al llarg del document. El primer és el token. Un model no rep necessàriament el text dividit paraula per paraula, sinó en unitats que poden ser paraules, fragments, signes o espais segons el sistema de tokenització.

Per això no existeix una equivalència exacta entre paraules i tokens. Com a orientació per a text en anglès, OpenAI utilitza l'aproximació de cent tokens per unes 75 paraules. En espanyol la proporció pot variar i també canvia amb el vocabulari, el codi o els números. Per a conèixer la grandària real d'un text cal utilitzar el tokenitzador corresponent, no comptar únicament paraules. [OpenAI, recompte de tokens]

La fitxa del model GPT Realtime 2.1 indica una finestra de context de 128.000 tokens. Si apliquem només l'aproximació anterior a text en anglès, obtenim unes 96.000 paraules: 128.000 x 0,75.

Per a posar en context (jaja), serien aproximadament 768 textos de 125 paraules que ATLAS podria tenir disponibles dins d'una conversa. Això donava un marge de treball espectacular per al projecte. [OpenAI, GPT Realtime 2.1]

Tampoc significa que tot aquest espai estigui reservat als documents de l'usuari. Les instruccions, la conversa i els resultats de les eines també formen part del que el sistema ha de manejar. En una sessió amb àudio, a més, no s'ha de convertir sense més tot el seu consum en un nombre de paraules escrites.

3.2. Context i límits de memòria

El context és la informació disponible per al model dins d'una interacció. Una comparació útil és imaginar una taula de treball: com més gran sigui, més documents es poden tenir oberts alhora. Però tenir un document sobre la taula no garanteix interpretar correctament cada detall ni recordar sempre on estava.

Aquesta distinció va corregir una idea massa optimista que havia plantejat al principi: una finestra gran no equival a un cervell que conserva cada paraula sense equivocar-se. El model pot confondre informació, no utilitzar una dada rellevant o respondre amb més seguretat de la que deuria. Per això les tasques importants necessiten eines i comprovacions, fins i tot quan la informació cap en el context.

Per a ATLAS, la capacitat disponible va permetre incloure bastant informació sobre l'usuari, el dispositiu i les funcions del projecte. No obstant això, no vaig voler omplir-la pel simple fet de poder fer-ho. Repetir una mateixa instrucció en diversos arxius augmenta la grandària i pot crear contradiccions quan s'actualitza només una còpia.

La solució va ser assignar responsabilitats als documents: un explica la identitat, un altre la personalitat, un altre les dades del projecte i uns altres les eines. Un manifest organitza què es carrega. Aquesta estructura resulta més fàcil de mantenir que una única explicació enorme on tot es barreja, i vaig heretar la idea principal d'OpenClaw.

3.3. Memòria entre sessions

La memòria persistent resol un problema diferent. En tancar una conversa, no n'hi ha prou amb confiar que el model recordarà el parlat en la següent. Si una dada ha de conservar-se, ha de guardar-se en un suport que el sistema pugui recuperar després.

En ATLAS vaig separar el coneixement relativament estable de la informació conversacional que canvia. El primer conté instruccions i manuals. La segona recull

estat i records que poden actualitzar-se durant l'ús. Així vaig poder revisar els documents del projecte sense tractar-los com si fossin un registre de tots els torns anteriors.

Per exemple, una preferència de l'usuari pot ser rellevant per a pròximes converses; la sortida completa d'una prova temporal o la sortida d'una ordre qualsevol, probablement no. Guardar indiscriminadament tot no sols ocupa espai: també fa més difícil trobar l'útil i augmenta la quantitat d'informació personal emmagatzemada.

Aquest mecanisme permet adaptació progressiva, però no canvia els paràmetres del model. El que es modifica és la informació que ATLAS consulta o rep. Mantenir aquesta diferència ajuda a descriure el projecte amb precisió i a no atribuir-li una forma d'aprenentatge que no s'ha implementat.

3.4. Eines i resultats verificables

Una eina és una interfície concreta entre el model i una capacitat externa. Pot consultar l'estat de la Raspberry Pi, obrir una aplicació o executar una rutina. El seu contracte defineix quines dades necessita, què retorna i quins errors poden aparèixer.

Vaig intentar que les operacions habituals tinguessin eines específiques. "Consultar ubicació" resulta més clar que lliurar una shell i esperar que l'agent descobreixi des de zero com parlar amb Android. Una funció estructurada permet validar paràmetres, distingir una falta de permís d'una desconnexió i evitar interpretar qualsevol sortida com un èxit.

El resultat torna a la conversa perquè ATLAS pugui continuar. Si obre una aplicació i la petició exigeix actuar dins d'ella, necessita una segona capacitat: observar i interactuar amb la seva interfície. Si una rutina obté una dada simple, pot no necessitar més raonament. L'eina adequada depèn de la tasca, no que l'agent vulgui utilitzar sempre l'opció més complexa.

L'autenticació d'un servei tampoc concedeix per si sola totes les capacitats d'un compte. Disposar d'una sessió vàlida no fa aparèixer automàticament Gmail, Drive o altres integracions en Realtime. Cada accés necessita una eina realment connectada i una autorització vàlida, al costat d'una adaptació que actue de pont entre aquesta i l'agent. Aquesta va ser una altra separació necessària entre el que un model podria fer i el que ATLAS tenia implementat.

4. Arquitectura pròpia d'ATLAS

4.1. Components i distribució de funcions

Després de la migració vaig organitzar ATLAS com un conjunt de components petits. El navegador s'ocupa de la interacció visible i de l'àudio; el servidor prepara sessions i eines; el sistema d'autenticació manté l'accés al proveïdor; i els serveis del dispositiu executen les operacions que els corresponen. Encara que l'usuari percebi un sol agent, internament hi ha diverses responsabilitats.

Aquesta separació va permetre diagnosticar millor les fallades. Una pantalla oberta no demostra que Realtime estigui connectat; una sessió Realtime activa no demostra que el telèfon estigui disponible. Abans, aquests estats podien barrejar-se en un missatge genèric de "connectant", que resultava poc útil tant per a l'usuari com per a mi.

La Raspberry Pi actua com a punt de coordinació. Allí es troben el context, les rutines, el servidor local i els serveis de l'A1. El model conversacional s'executa en el núvol. Per tant, el projecte utilitza una arquitectura híbrida: part del treball és local i una altra part depèn de serveis externs. No seria correcte presentar-ho com un assistent completament offline.

Un avantatge d'aquesta organització és que una interfície pot canviar sense haver de refer tota la lògica. La pantalla minimalista i la vista de depuració, per exemple, reutilitzen el mateix flux de conversa. No vaig voler mantenir dos agents que es comportessin de manera diferent per utilitzar dissenys diferents. Les responsabilitats de cada component es poden consultar en l'annex A, apartat A.1.

4.2. Native Broker i preparació de les sessions

Vaig anomenar Native Broker al component que concentra la relació amb l'autenticació i la preparació de Realtime. En la implementació del prototip utilitza el servidor d'aplicació de Codex i conserva separat el seu estat d'accés. D'aquesta forma, el navegador no necessita rebre la credencial persistent del compte.

Aquesta ruta d'accés es va utilitzar en el meu entorn de desenvolupament, però no suposa que una subscripció de ChatGPT o Codex concedeixi, de manera general, ús il·limitat de qualsevol API. La disponibilitat, les quotes i les condicions depenen del servei contractat. La distribució del producte requereix una via d'accés suportada i apropiada per a cada usuari.

També vaig aprendre a separar errors d'autenticació d'errors de transport o del proveïdor. Un missatge de servidor 500 no demostra que s'hagi corromput l'inici de sessió. Tornar a autenticar pot ser necessari en alguns casos, però fer-ho sense comprovar els registres només canvia una peça i pot deixar intacte el problema real.

Preparar una sessió té un cost diferent al de respondre dins d'una ja preparada. Es negocia la connexió, es configura la veu i es carrega el context. Per això vaig procurar no repetir aquest procés en cada pulsació del micròfon. L'objectiu era que la conversa estigués disponible en necessitar-la, sense mantenir recursos costosos actius quan no aportaven res.

4.3. Organització d'arxius i context

La carpeta `.atlas` reuneix el programari i les dades del projecte, però no tot es guarda barrejat. Dins de `context/knowledge` es troben els documents de coneixement i instruccions. En `context/conversation` es conserva l'estat conversacional mutable. Els serveis, les rutines, els perfils i la configuració privada tenen espais propis.

Els noms d'alguns documents fan explícita la seva responsabilitat. `IDENTITY.md` descriu qui és ATLAS; `SOUL.md`, el seu comportament; `USER.md`, informació rellevant de l'usuari; i `TOOLS.md`, una guia de capacitats. `AGENTS.md` relaciona les diferents peces. Un manifest estableix l'ordre de càrrega i evita incorporar dues vegades un mateix arxiu.

Per al treball de recerca vaig aplicar el mateix criteri. `TDR.md` conté les dades i terminis particulars del projecte. `TDRMAN.md` resumeix les normes de l'institut. La transcripció completa del manual es conserva apart i es consulta quan fa falta, sense afegir totes les seves pàgines a cada torn de conversa.

L'estructura no pretén ser una memòria il·limitada. És una forma controlable de decidir quina informació rep l'agent. La recuperació selectiva de documents evita carregar informació innecessària i permet comprovar que no es perd l'important. Afegir arxius sense revisar el seu contingut no és una estratègia suficient de gestió del context.

4.4. Rutines per a tasques repetitives

No totes les tasques necessiten que el model raoni de nou. Si li demano l'hora, el sistema pot executar una consulta local i convertir el resultat en una frase. Fer que un

agent explori diverses eines per a resoldre això afegeix temps i oportunitats d'error sense aportar un avantatge real.

Per a aquestes situacions vaig preparar un sistema de rutines. Cada rutina té un nom, una o diverses frases d'activació i passos definits. Pot executar un ordre, guardar la seva sortida en una variable i utilitzar-la en una resposta. El registre inclou si el resultat necessita interpretació del model o si pot retornar-se directament.

En una rutina d'hora, per exemple, no es guarda una resposta fixa com "són les cinc". Es guarda com obtenir l'hora actual i com inserir-la en la frase. Així, l'execució és repetible sense deixar de dependre d'una dada real del sistema. La veu encara necessita sintetitzar aquesta frase, per la qual cosa executar l'ordre en mil·lisegons no garanteix sentir-la en aquest mateix temps.

La creació o modificació d'una rutina sí que pot requerir a l'agent. Primer cal entendre què vol l'usuari, identificar el dispositiu correcte i comprovar que el procediment funciona. Una vegada resolta aquesta part, l'ús quotidià pot seguir una ruta més directa. Si falla, es retorna l'error per a analitzar-lo sense repetir a cegues una acció amb efectes.

4.5. Perfils i límits de capacitats

També vaig preparar una estructura declarativa de perfils, als quals vaig anomenar rols. La idea és que no totes les instal·lacions necessitin el mateix coneixement ni les mateixes eines. Un dispositiu de consulta escolar, per exemple, no hauria de rebre accés administratiu només perquè el prototip personal el té.

El perfil habitual és atlas-full. Inclou les capacitats completes del prototip i, per a les proves, informació de professors. També vaig preparar manifestos de perfils més limitats per a estudiar altres usos. Un manifest descriu les capacitats d'un perfil, però el seu aïllament depèn dels controls aplicats per programari.

Aquesta distinció és important perquè una instrucció escrita no és una barrera de seguretat. Indicar "no utilitzis la shell" en un document pot orientar al model, però la protecció real consisteix a no oferir aquesta eina o rebutjar la seva execució des del programa que la controla.

L'exemple de professors va permetre estudiar com respondre a partir d'un context fitat i de l'hora actual, sense convertir una consulta senzilla en una recerca llarga. La

informació de professors i horaris es va utilitzar com a context de prova, no com un desplegament oficial de l'institut.

5. Interfícies i connexió amb altres dispositius

5.1. WebScreen i conversa per veu

WebScreen és la pàgina que mostra ATLAS A1. Utilitza HTML, CSS i JavaScript per a la interfície i un servidor local per a les funcions del dispositiu. La veu en temps real es transporta mitjançant WebRTC, mentre que altres missatges de control i resultats d'eines es coordinen amb el servidor.

La paraula d'activació és "ATLAS". El controlador decideix quan una intervenció pot iniciar un torn i quan ha d'ignorar-se. Aquesta decisió no pot dependre només de demanar al model que esperi: si ja s'ha obert una resposta, el problema de coordinació ha ocorregut abans que la instrucció de personalitat pugui resoldre'l.

Durant el desenvolupament vaig provar mantenir una finestra d'escolta després de respondre. En una primera revisió la vaig retirar per a evitar que ATLAS incorporés frases dirigides a una altra persona. Més endavant vaig afegir una finestra de 3 segons: en acabar de parlar, continua escoltant i permet continuar sense repetir el seu nom. Si començo una frase dins d'aquest interval, em deixa acabar-la; si no, torna a esperar la paraula d'activació. El temps compta des del final real de l'àudio.

La interfície té una vista tècnica amb registres i una altra minimalista per a l'ús diari. Les expressions i animacions indiquen estats i fan a l'agent més pròxim; no modifiquen per si mateixes el control de la conversa. També vaig incorporar ajustos locals de xarxa i un teclat tàctil anomenat "teclao" (em semblava graciós el nom jaja), per a poder connectar la Raspberry Pi a un altre Wi-Fi des de la seva pantalla.

La pàgina local pot mostrar-se encara que no hi hagi Internet, sempre que el servidor i el navegador de l'A1 funcionin. Això no significa que el model o tots els serveis de veu funcionin sense xarxa. Distingir totes dues coses és essencial: una interfície disponible pot servir per a recuperar la connexió, encara que la conversa no estigui preparada.



Figura 2. Prototip real amb pantalla i micròfon. Fotografia pròpia retocada per a eliminar reflex i pols i substituir el rellotge per la cara habitual.

5.2. Veu nativa i motors alternatius

Amb Realtime, la ruta de veu nativa permet que el model produeixi àudio com a part de la seva resposta. Ja no és obligatori esperar a un text complet, enviar-lo a un altre servei i rebre després un arxiu de veu. Aquesta va ser una de les raons principals per a canviar d'arquitectura.

No obstant això, vaig conservar opcions com ElevenLabs i la síntesi del navegador. Podien interessar-me pel seu timbre o per la disponibilitat d'una veu concreta. En aquests casos sí que existeix una etapa addicional de síntesi. Dir que Realtime va eliminar tota transcripció i tot motor extern en qualsevol configuració seria simplificar massa: va eliminar la necessitat d'aquesta cadena en la ruta nativa.

Per a reduir esperes vaig treballar amb respostes progressives o streaming. En lloc d'esperar al resultat complet, el programa rep fragments i comença a processar-los quan té informació suficient. Això exigeix cuidar els límits de les frases, la reproducció i les cancel·lacions; no n'hi ha prou amb dividir arbitràriament un text en trossos.

La documentació de ElevenLabs distingeix diferents factors de latència, i el mesurament del prototip va confirmar que calia observar més d'un instant. Rebre el primer byte d'àudio és diferent que el navegador pugui reproduir-lo, i també d'escoltar-lo per l'altaveu. Aquesta diferència apareix de nou en l'avaluació. [ElevenLabs, optimització de latència]

5.3. El xat de terminal com a eina de proves

Vaig crear `atlas-chat` per a poder interactuar per text amb la mateixa base conversacional i d'eines. Em permetia repetir una petició sense dependre del micròfon, comprovar quin ordre havia sol·licitat el model i veure la seva sortida. Així podia separar un error de lògica d'un problema d'àudio o reconeixement.

El mode interactiu manté una conversa i el paràmetre `-p` permet enviar una petició des de la línia d'ordres. La interfície diferencia les respostes de l'agent de l'activitat tècnica, i inclou historial, edició d'entrada i suggeriments. Són ajudes per a depurar, no capacitats noves del model.

També disposa d'un mode de prova que no incorpora els torns a la memòria conversacional compartida. Això evita que una sèrie d'assajos artificials es converteixi en records habituals de l'usuari. No ha de confondre's amb una garantia d'absència de registres: un diagnòstic pot continuar deixant traces tècniques que necessiten la seva pròpia política de conservació.

Una prova satisfactòria en terminal demostra que cert recorregut de text i eines funciona. No demostra que el micròfon detecti bé la paraula ATLAS, que no hi hagi ressò o que l'altaveu reprodueixi la resposta. La terminal va resultar útil precisament perquè permet aïllar aquesta part del problema.

5.4. Aplicació Android i servei Companion

L'aplicació Android va ampliar l'accés a l'agent anés de la pantalla de l'A1. Vaig organitzar les seves funcions en conversa, accions, terminal, estat i ajustos. L'objectiu era poder parlar amb ATLAS, executar accions preparades i consultar el dispositiu des del telèfon, mantenint una experiència coherent.

En la Raspberry Pi, el servei Companion gestiona la relació amb l'aplicació. La connexió de control i la sessió de veu són diferents: mantenir el telèfon vinculat no obliga a mantenir el micròfon ni WebRTC oberts tot el temps. Aquesta separació va ser especialment important en optimitzar el consum en segon pla.

L'enllaç de control utilitza un servei visible per a Android i pot mantenir-se en minimitzar l'aplicació, mentre el sistema operatiu permeti executar el procés. No és correcte garantir una connexió eterna: Android, el fabricant, una pèrdua de xarxa o una ordre de forçar detenció poden interrompre-ho. L'aplicació ha de reconèixer l'estat i recuperar-se, no fingir que segueix connectada.

La terminal també es gestiona com una sessió pròpia. Pot sobreviure a una recreació de la pantalla de l'aplicació i tancar-se després d'un període d'inactivitat. En l'estat del dispositiu vaig conservar l'última dada vàlida mentre se sol·licitava un nou, perquè una recàrrega no deixés la interfície buida. Mostrar una dada anterior exigeix distingir-lo d'una lectura recent quan la connexió falla.

Els permisos se sol·liciten explicant la seva finalitat. Alguns són necessaris per a una funció concreta i uns altres són opcionals. Donar permís al micròfon no autoritza a accedir a contactes; concedir accessibilitat tampoc significa que l'agent hagi d'utilitzar-la per a qualsevol operació. La interfície de permisos forma part del control que conserva l'usuari.

5.5. Aparellament i transport privat

Inicialment vaig provar una connexió basada en un relay extern. Més endavant vaig passar a Tailscale per a connectar el portàtil, la Raspberry Pi i el telèfon dins d'una xarxa privada. Aquesta decisió buscava simplificar l'accés entre els meus dispositius i afavorir una ruta directa quan fos possible.

Tailscale intenta establir connexions directes i pot recórrer a un relay quan les condicions de la xarxa ho impedeixen. Estar en la mateixa Wi-Fi sol afavorir l'enllaç directe, però no convé assumir-lo sense comprovar-lo. Un túnel privat tampoc elimina la latència dels servidors del model ni la d'una captura de pantalla. [Tailscale, tipus de connexió]

Vaig mantenir un aparellament propi d'ATLAS. L'A1 obre una finestra temporal i mostra un codi de sis xifres; el telèfon l'introdueix durant el procés de vinculació. El codi facilita un lliurament autoritzat de credencials, però no substitueix la clau que protegirà les sessions posteriors.

La pertinença a la xarxa privada i l'aparellament són capes diferents. Que un equip aconsegueixi la direcció de l'A1 no significa que hagi de poder controlar-ho. El transport i els missatges es protegeixen amb mecanismes de xifrat, i el desaparellament invalida la relació perquè no basti amb conservar una pantalla antiga de l'aplicació.

5.6. Control natiu i control visual del telèfon

Vaig dividir el control del telèfon en dos recorreguts. El primer utilitza funcions natives: obrir una aplicació o consultar informació autoritzada mitjançant una operació definida.

El segon, Android Use, interactua amb la pantalla quan no existeix una funció directa adequada.

Aquesta diferència evita utilitzar captures i coordenades per a tot. Si el sistema sap obrir una aplicació pel seu identificador, no necessita recórrer visualment el llançador. Però si la petició continua dins d'aquesta aplicació, pot ser necessari prendre una captura, llegir elements accessibles i realitzar gestos. Obrir l'aplicació no ha de confondre's amb acabar tota la tasca.

Android Use segueix un cicle d'observació, acció i comprovació. Primer obté informació de la pantalla; després executa una acció o un petit lot d'accions previsibles; finalment comprova el nou estat. El lot redueix intercanvis innecessaris, però no ha de ser tan llarg que continuï actuant sobre una pantalla diferent de l'esperada.

Quan hi ha etiquetes accessibles fiables, es pot prémer un element per la seva descripció. Quan no n'hi ha, les coordenades continuen sent útils. Totes dues estratègies necessiten una observació recent: una captura antiga no garanteix que el botó continuï en el mateix lloc després de carregar contingut, obrir el teclat o canviar d'orientació.

El servei d'accessibilitat permet les operacions corresponents dins dels límits d'Android i ha de ser habilitat per l'usuari. Mentre ATLAS controla visualment el telèfon es mostra una indicació i una forma clara de detenir-lo. No s'intenten eludir les pantalles protegides ni convertir el permís en un accés ocult. [Android Developers, AccessibilityService]

Les proves de consultes i navegació no equivalen a validar crides o missatges reals. Aquestes accions tenen efectes sobre altres persones i requereixen condicions d'assaig diferents. En la documentació vaig separar les capacitats implementades, els permisos disponibles i les operacions efectivament comprovades. Les comprovacions realitzades amb Android es recullen en l'annex B, apartat B.5.

5.7. Integració de WhatsApp

Per a utilitzar ATLAS des de WhatsApp vaig integrar Baileys i vaig preparar un servei propi en la Pi. Permet vincular el meu compte amb un QR, consultar missatges, respondre i programar tasques. Vaig separar dues maneres: Guardian, per a parlar amb ATLAS, i Sami absent, per a respondre en les converses que jo selecciono.

Guardian respon quan escric "@atlas" des del meu compte. Rep el nom i el tipus de xat, fins a 30 missatges recents i, si estic responent a algú, el missatge citat. Després només s'afegeixen els missatges que encara no ha rebut. Manté la sessió preparada i conserva les eines d'ATLAS, de manera que puc demanar-li accions com en el xat del dispositiu.

En Sami absent vaig preparar un perfil de la meva manera d'escriure i un índex privat d'exemples. No vaig entrenar un model nou: vaig utilitzar instruccions i una cerca d'exemples. Com que aquest mode rep missatges d'altres persones, no té shell ni accés general al sistema; només pot consultar aquest índex de lectura. Pot respondre en diverses bombolles, citar missatges o no contestar quan no fa falta.

També vaig afegir recordatoris amb un missatge fix o una instrucció per al model, com resumir un grup. El servei controla el destinatari i l'horari. Un monitor mostra les accions i els seus temps per a localitzar fallades. Les credencials i converses personals no es publiquen en GitHub. La connexió, les dues maneres de conversa i les automatitzacions s'expliquen en l'annex E, apartats E.1 a E.4.

5.8. Accés des de la web

La web atlas.samilososami.com presenta el projecte. També inclou dos accessos protegits amb contrasenya: "/try", per a provar la interfície del WebScreen, i "/chat", per a conversar per escrit. El xat permet guardar diverses converses en el navegador, adjuntar imatges i veure la resposta mentre es genera. Tots dos accessos comparteixen la sessió, per la qual cosa no fa falta introduir la contrasenya dues vegades.

El portal funciona com un servei Python en la Raspberry Pi, accessible mitjançant HTTPS i Tailscale Funnel. La contrasenya es guarda com un hash amb sal i el navegador conserva una cookie de sessió revocable. Aquesta protecció era necessària perquè el portal dona accés a les capacitats reals d'ATLAS. L'historial es guarda en el navegador, encara que els missatges necessaris per a respondre s'envien a la Pi i al proveïdor del model.

Vaig preparar instruccions específiques per a la conversa escrita. Per exemple, el xat mostrava "48,3 °C" en lloc d'escriure el número complet amb paraules, com ocorria amb la veu. Els avisos d'eines apareixen al costat de la resposta. Cada xat té el seu

propi historial, separat de la pantalla física i de WhatsApp. Els detalls d'autenticació, historial i separació de sessions es poden consultar en l'annex E, apartats E.5 i E.6.

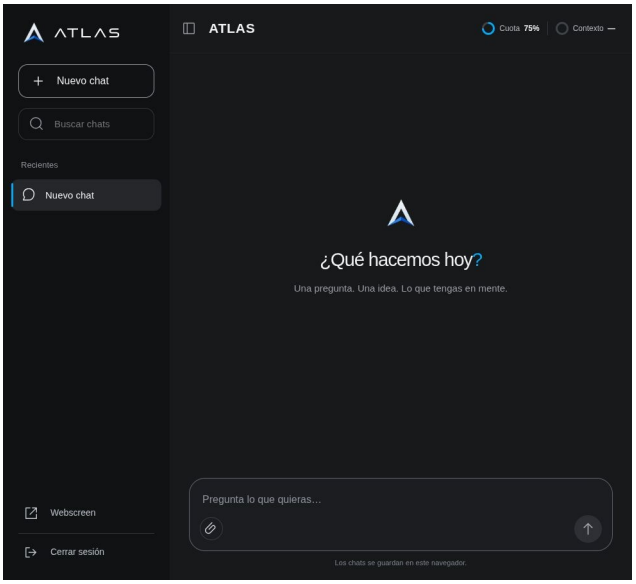


Figura 3. Interfície real de /chat, sense converses personals. Captura pròpia del portal, 18 d'octubre de 2026.

5.9. Expressivitat de la interfície

A partir dels ulls blaus i el somriure d'ATLAS vaig preparar un catàleg de 323 expressions: les 23 que ja tenia i dues col·leccions noves de 150. Vaig combinar parpelles, celles, boques i alguns accessoris, mantenint les proporcions i el gruix dels traços. Les imatges generades van servir com a referència; la interfície dibuixa les cares mitjançant vectors.

Les expressions tenen transicions d'entrada i sortida i comparteixen set gestos bàsics. Per tant, són 323 poses, no 323 animacions independents. Vaig limitar els canvis automàtics a dos per resposta i els vaig coordinar amb la veu perquè ATLAS pogués expressar-se sense canviar de cara constantment. El catàleg i el seu funcionament s'amplien en l'annex F, apartat F.1.

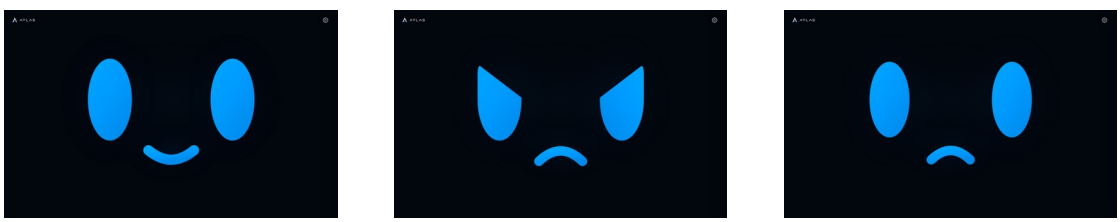


Figura 4. Tres expressions d'ATLAS: neutral, enfadat i trist. Captures del renderitzador vectorial. Elaboració pròpia.

6. Fiabilitat, seguretat i manteniment

6.1. Control de torns i recuperació de connexions

Una part considerable del treball va ser aconseguir que una conversa no es trenqués davant esdeveniments petits. El navegador podia notificar un canvi temporal de connexió, arribar un fragment tardà de transcripció o completar-se una resposta just quan s'intentava cancel·lar-la. Tractar tots aquests casos com a errors greus provocava reconnexions innecessàries.

Per això vaig separar l'estat de transport de l'estat del torn. No és el mateix que la xarxa estigui momentàniament inestable, que la sessió hagi caducat o que un usuari interrompi una resposta. Cada situació necessita una reacció diferent. Reiniciar tot indiscriminadament pot esborrar el context temporal d'una intervenció que encara estava en curs.

També va ser necessari relacionar els esdeveniments amb la seva intervenció concreta. Un senyal que va acabar un àudio anterior no ha de tancar el torn que acaba de començar. De la mateixa manera, demanar una cancel·lació quan ja no hi ha resposta activa no hauria de provocar per si sol una nova connexió completa.

La recuperació millora l'experiència, però no pot mantenir una connexió física que ja no existeix. Davant una interrupció real, l'objectiu és detectar la fallada, conservar el recuperable i restablir el servei sense repetir accions amb efectes. La interfície no ha de prometre que no va haver-hi cort si es va perdre part d'una petició.

6.2. Soroll, interrupcions i naturalitat

Un agent de veu rep molt més que frases netes. Hi ha soroll de fons, converses pròximes, el so del seu propi altaveu i pauses dins d'una intervenció. Si qualsevol variació de volum canvia l'estat entre escoltar i pensar, la interfície sembla indecisa i el model pot rebre fragments solapats.

Vaig treballar en llindars d'entrada i en l'estabilitat d'aquests estats, buscant un equilibri entre sensibilitat i falses activacions. Una veu molt baixa no té per què iniciar el mateix recorregut que una petició dirigida clarament al dispositiu. No obstant això, un llindar fix massa alt també pot excloure a un usuari que parla suaument o està més lluny. El calibratge té límits i necessita proves acústiques reals.

La naturalitat tampoc es resolia afegint frases suposadament humanes. Respondre "consultaré el meu context" a una pregunta senzilla explicava un procés intern que l'usuari no necessitava escoltar. Vaig preferir instruccions per a respondre de manera breu, reconèixer el que faltava i evitar preàmbuls automàtics quan la dada podia donar-se directament.

Una expressió animada pot acompanyar aquesta resposta, però no ha de controlar quan s'admet veu ni quant triga a executar-se una eina. Mantenir la presentació separada del controlador va ser important per a poder millorar l'aparença sense introduir corts en la conversa.

6.3. Dades privades i permisos

ATLAS maneja informació personal i eines que poden modificar dispositius. Les credencials de serveis, les dades d'aparellament i el context de l'usuari no han de tractar-se com a arxius públics del projecte. Vaig separar aquests elements dels materials que es poden versionar i documentar.

Les claus es guarden amb accés restringit i no haurien d'aparèixer en captures, sortides d'ordres ni imatges distribuibles. Els registres també necessiten cura: poden incloure transcripcions o informació d'una pantalla. Que siguin útils per a depurar no justifica conservar-los indefinidament o pujar-los complets a un repositori.

Les instruccions de prudència ajuden, però no substitueixen controls tècnics. Una funció pot limitar quina operació accepta, requerir permís o retornar un error si el telèfon no està disponible. Les accions destructives necessiten especial atenció al seu objectiu i reversibilitat. El model no ha d'interpretar un contingut oposat en una pàgina com una nova autorització del propietari.

La utilitat de l'agent augmenta amb les seves capacitats, però també ho fa la responsabilitat de limitar el seu abast.

6.4. Diagnòstic i manteniment

Vaig preparar ordres per a revisar l'estat general, la connexió de l'aplicació i els serveis. atlas-rafas concentra comprovacions i operacions de diagnòstic; atlas-status resumeix el sistema; i els registres permeten reconstruir una seqüència d'esdeveniments. El catàleg abreujat d'ordres es pot consultar en l'annex A, apartat A.2.

Així vaig poder distingir fallades de xarxa, d'àudio o de Companion sense atribuir-los tots al model. Vaig utilitzar systemd per a gestionar els serveis, Git per a conservar canvis i GitHub per a distribuir les versions. Poder tornar a una versió coneguda va resultar tan important com introduir funcions noves.

No totes les fallades han de reparar-se automàticament. Reiniciar l'equip, canviar de xarxa o modificar credencials pot interrompre treball vàlid. El diagnòstic ha de distingir entre observar i modificar, i conservar informació suficient per a entendre què va ocórrer.

II. ELECTRÒNICA — El dispositiu ATLAS A1

7. Selecció de components i integració física

7.1. Selecció de la Raspberry Pi

Fins a aquest punt, ATLAS podia entendre's com un conjunt de programes. No obstant això, la meva idea inicial incloïa un dispositiu dedicat: alguna cosa que pogués estar en una taula, escoltar i respondre sense dependre de mantenir obert el meu portàtil. ATLAS A1 va ser la primera materialització d'aquesta idea.

Vaig triar una Raspberry Pi 5 amb 4 GB de memòria RAM. Disposa d'un processador ARM de quatre nuclis a 2,4 GHz i connexions suficients per a pantalla, àudio i xarxa. La seva grandària i la documentació disponible van facilitar el muntatge d'un prototip ampliable. Les característiques tècniques de referència procedeixen del fabricant; l'ús que els vaig donar correspon al desenvolupament d'ATLAS. [Raspberry Pi, Raspberry Pi 5]

L'elecció va ser un compromís. Un ordinador més potent hauria donat més marge per a models locals, però també hauria canviat la grandària, el consum i el cost del dispositiu. Un microcontrolador hauria estat apropiat per a llegir sensors o controlar sortides senzilles, però no per a executar còmodament el navegador i tot l'entorn d'ATLAS.

En aquest disseny, la Raspberry Pi no executa el model conversacional gran. Coordina serveis locals i es comunica amb el proveïdor en el núvol. Això permet utilitzar un ordinador petit, a canvi de dependre de la connectivitat i del servei extern. L'elecció del maquinari i l'arquitectura de programari no van ser decisions independents.

Els quatre gigaoctets tampoc són un espai exclusiu per a ATLAS. Es comparteixen entre sistema operatiu, navegador, àudio i processos auxiliars. Per això vaig haver de parar esment al nombre de sessions, als recursos en segon pla i a quins serveis valia la pena mantenir actius.

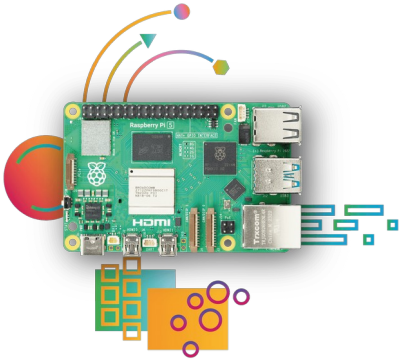


Figura 5. Raspberry Pi 5, placa utilitzada com a base de l'A1. Font: Raspberry Pi, fitxa oficial del producte.

7.2. Pantalla i entrada tàctil

Vaig utilitzar una pantalla SunFounder TS-7 Pro de set polzades, amb resolució de 1024 × 600 píxels. Rep la imatge per HDMI i comunica el tacte per USB; també incorpora altaveus. Tenir diverses funcions en un mateix conjunt va reduir la quantitat de mòduls que necessitava afegir al prototip. [SunFounder, Hardware introduction]

La pantalla va condicionar la interfície. En set polzades, omplir l'espai d'indicadors i registres pot ser útil per a depurar, però no per a parlar amb un agent. Per això vaig mantenir la vista tècnica i vaig preparar una altra molt més senzilla per a l'ús normal. El disseny havia de continuar sent llegible a certa distància i permetre reconèixer l'estat sense estudiar un panell.

Durant les primeres proves va aparèixer un cas que va ajudar a separar maquinari i programari: es veia l'arrencada i després la pantalla mostrava que no rebia senyal. Això no significava necessàriament que el cable o el panell estiguessin avariats. El sistema necessitava mantenir una sessió gràfica apropiada després de la fase inicial.

Una altra dependència va aparèixer amb el mode ocult. Apagar físicament la pantalla podia fer desaparèixer també la sortida d'àudio que utilitzaven els seus altaveus. Per a poder continuar parlant amb ATLAS sense mostrar la interfície, vaig preparar un mode de pantalla negra i lluentor mínima que manté l'enllaç necessari. És una ocultació visual, no un apagat elèctric complet.

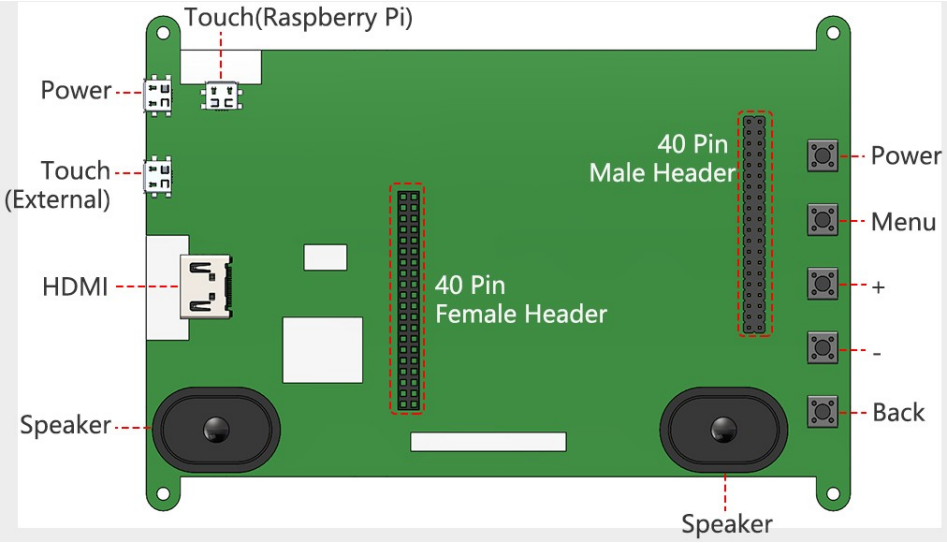


Figura 6. Connexions, altaveus i quatre ancoratges de la pantalla TS7 Pro. Font: SunFounder, Hardware Introduction.

7.3. Micròfon, altaveus i Bluetooth

L'entrada de veu del prototip utilitza un micròfon USB que Linux identifica com Jieli UACDemoV1.0. Treballa amb un canal mono i una freqüència de mostreig de 48 kHz. El sistema pot detectar la seva connexió com la de qualsevol altre perifèric USB, però reconèixer el dispositiu no demostra que el navegador estigui capturant correctament.

La col·locació del micròfon i dels altaveus influeix directament en la conversa. Si ATLAS sent la seva pròpia veu, pot confondre-la amb una nova intervenció. Les proteccions de programari ajuden, però no substitueixen un bon disseny acústic. Per aquest motiu vaig separar les entrades i sortides d'àudio i vaig tractar el retorn acústic com una limitació específica del prototip.

També vaig provar una barra de so Bluetooth. En aquest procés vaig comprovar que aparellar i reproduir àudio són operacions diferents. El dispositiu podia figurar com a conegut i, no obstant això, no disposar del perfil adequat per a la reproducció. Va ser necessari revisar la relació entre BlueZ, el gestor d'àudio i la sessió d'usuari.

PipeWire i WirePlumber intervenen en la selecció de destinacions de so. La verificació de la sortida combina la selecció de la destinació i el volum amb la comprovació de l'altaveu real. Un to de prova sentit físicament proporciona una evidència diferent d'una línia en el registre.

Aquestes proves van mostrar que el temps de resposta no depèn només de la inferència del model. Si l'àudio s'envia a una destinació incorrecta, la conversa sembla

no respondre encara que el text ja estigui complet. El diagnòstic ha de recórrer tota la cadena, des de la captura fins a la reproducció.

7.4. Alimentació, emmagatzematge i refrigeració

L'alimentació havia de sostenir la Raspberry Pi i els perifèrics connectats. El fabricant recomana una font USB-C adequada per a la Pi 5, capaç de subministrar 5 V i 5 A. Aquesta referència no permet deduir per si sola el consum constant del muntatge: la demanda canvia segons la càrrega i els dispositius utilitzats. [Raspberry Pi, Raspberry Pi 5]

Durant part del desenvolupament vaig utilitzar alimentació portàtil. Quan una bateria s'esgotava, l'equip podia apagar-se de manera brusca i després semblar una fallada del programa. Afegir informació sobre arrencades i apagats va ajudar a distingir una ordre normal d'apagat d'una interrupció inesperada.

La refrigeració es va realitzar mitjançant un Active Cooler. Mantenir la temperatura controlada és important tant per al maquinari com per a interpretar les proves: si el processador redueix la seva freqüència per calor, canvia el rendiment encara que el codi sigui el mateix. El disseny de la carcassa incorpora reixetes de ventilació per a facilitar el flux d'aire.

L'emmagatzematge conté el sistema, les aplicacions, els arxius de context i els registres. Aquests últims poden créixer durant la depuració i necessiten una política de conservació. També cal distingir la capacitat nominal del suport de l'espai disponible en una partició: el sistema d'arxius i les dades existents redueixen el que queda lliure.

Per a les actualitzacions habituals és més raonable distribuir els arxius modificats que generar una còpia completa del disc. Una imatge del sistema és útil per a instal·lar una base coneguda, però ocupa més, triga a preparar-se i exigeix revisar que no inclogui credencials, xarxes privades o dades personals del prototip.

8. ATLAS OS, recuperació i disseny de la carcassa

8.1. Adaptació del sistema operatiu

Vaig anomenar ATLAS OS a l'adaptació de Linux que vaig preparar per a l'A1. Està basada en l'entorn Debian i Raspberry Pi utilitzat pel dispositiu. No vaig desenvolupar un kernel, controladors ni un gestor de paquets propis; vaig aprofitar aquesta base i vaig personalitzar els serveis, l'arrencada, la identitat i les maneres d'ús.

Aquesta decisió responia a l'objectiu del projecte. Escriure un sistema operatiu complet no millorava de manera raonable la conversa ni les eines de l'agent. En canvi, l'arrencada en la interfície adequada i la recuperació davant fallades aporten valor directe.

Els serveis s'inicien i supervisen per separat. La pantalla pot canviar de mode sense que per això calgui esborrar el context; l'enllaç amb Android pot recuperar-se sense reconstruir tota la interfície. El nom i els elements visuals d'ATLAS OS donen coherència al dispositiu, però no oculten la base tècnica de la qual depèn.

Vaig preparar també una imatge distribuible del sistema. La imatge exclou secrets i dades de la meva instal·lació personal, i la seva primera arrencada permet que cada usuari configuri els seus propis accessos sense heretar els del desenvolupador.



Figura 7. Escriptori d'ATLAS OS en la pantalla del prototip durant el desenvolupament. Fotografia pròpia.

8.2. Modes d'ús i recuperació local

L'ordre `atlas-screen` permet alternar entre la interfície d'ATLAS, l'escriptori, la terminal i la recuperació. La vista nova i la de depuració comparteixen la base conversacional. El mode ocult conserva la conversa i l'àudio sota una imatge negra, mentre que l'apagat de pantalla té un altre efecte sobre el maquinari.

Aquestes opcions no són solament comoditats per a desenvolupar. Si la xarxa o el proveïdor fallen, el propietari necessita una manera de continuar treballant amb l'equip. La pàgina local permet gestionar la connexió Wi-Fi i el teclat tàctil facilita introduir dades sense dependre d'un teclat físic per a cada operació.

Per a les fallades més greus vaig preparar RAFAS, Recovery Access For ATLAS Systems. Proporciona una ruta local de recuperació que no depèn del fet que el model entengui una petició ni que WebScreen funcioni. El seu propòsit és retornar el control a una persona quan les capes superiors no estan disponibles.

En el prototip, la recuperació física es va configurar amb facilitats administratives pensades per a les meves proves. Aquest mode permet accedir directament al manteniment local del dispositiu.

8.3. Disseny digital de la carcassa

Vaig dissenyar la carcassa en FreeCAD amb una forma triangular i vores arrodonides. La cara frontal i la posterior tenen una inclinació de 30° respecte a la vertical, amb un vèrtex teòric de 60°. Volia un acabat negre senzill, amb la pantalla com a element principal. Les primeres imatges em van ajudar a definir l'aspecte, però per a crear el model vaig haver de tenir en compte les mesures dels components i l'espai dels connectors.

A principis d'estiu vaig imaginar una primera proposta visual d'ATLAS Home V1. Aquesta imatge em va servir com a inspiració del producte, abans de triar i muntar els components. La seva pantalla de 5 polzades i la indicació de IA local pertanyen a aquella idea inicial: el prototip final utilitza una pantalla de 7 polzades i models externs.



Figura 8. Concepte inicial que vaig imaginar en començar el treball i que em va inspirar a principis d'estiu de 2026. Imatge conceptual, no model CAD ni muntatge final.

Vaig anar ajustant la distribució interna. Vaig desplaçar la Raspberry Pi cap a dins per a deixar espai al HDMI, al USB del tàctil i al cable del micròfon. Aquest últim es conserva sencer en el lateral dret, separat dels altaveus frontals. També vaig deixar ventilació, espai per a recollir el cable sobrant i buits per a portar els ports a la part posterior mitjançant extensors.

En R04 vaig reduir la carcassa a 190 × 163,2 × 144 mm, sense comptar els peus ni el sortint del polsador. Vaig mantenir la pantalla real de 7 polzades i vaig reduir el marc respecte a R03. Vaig preparar vistes frontal, lateral i superior, seccions i un especejament per a veure com encaixaven les peces. La distribució interna i els plans es poden consultar en l'annex D, apartats D.2 i D.5.



Figura 9. Disseny d'ATLAS A1 amb la carcassa negra. Render del model 3D.

El muntatge està dividit en quatre peces: cos amb base, frontal, tapa posterior i cèrcol del micròfon. S'uneixen amb caragols i rosques captives per a poder obrir-ho quan sigui necessari. La Pi utilitza caragols M2,5 i les tapes M3. La pantalla manté els seus quatre ancoratges. L'encesa es resol en el disseny amb un polsador momentani connectat a J2 de la Pi.

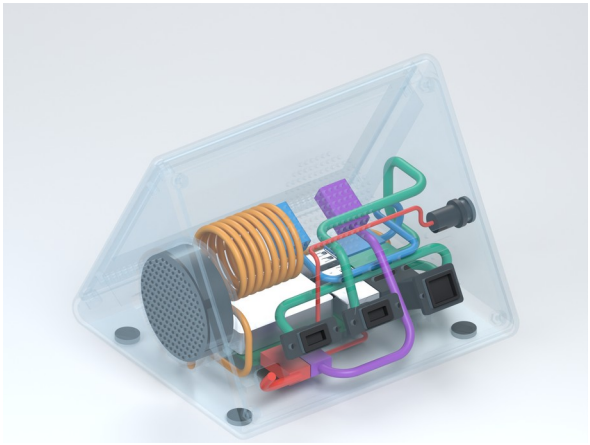


Figura 10. Distribució interna i reserves de cablejat del CAD R04. Vista amb la carcassa transparent. Elaboració pròpia.

8.4. Configuració per a Bambu Lab H2S

La impressora de l'institut és una Bambu Lab H2S, amb un volum d'impressió de 340 × 320 × 340 mm. Encara que permet diversos colors, vaig preparar la carcassa completament en negre. Vaig utilitzar Bambu Studio per a dividir les peces en capes i calcular els recorreguts d'impressió. [Bambu Lab, H2S]

En R05 vaig preparar un perfil de PETG negre, broquet de 0,4 mm i capes de 0,20 mm, amb quatre parets, cinc capes superiors i inferiors i un farcit gyroid del 20%.

El disseny va quedar dividit en quatre peces i sis provetes per a verificar els encaixos en el model digital. Bambu Studio va estimar 15 h 47 min i 367,1 g per a la carcassa, sense comptar les provetes. El laminatge, les peces i el muntatge es detallen en l'annex D, apartats D.3 i D.4.

8.5. Pressupost del prototip

El cost directe aproximat del prototip va ser de 225 €. La xifra inclou els components principals que vaig necessitar per a construir ATLAS A1. La impressió 3D no va tenir cost per a mi perquè vaig utilitzar la impressora i el filament de l'institut.

Element	Procedència	Cost aproximat
Impressió 3D	Impressora i filament de l'institut	0 €
Micròfon USB	Comprat per al prototip	15 €
Raspberry Pi 5 de 4 GB i Active Cooler	Comprats per al prototip	125 €
Pantalla tàctil SunFounder TS7 Pro	Comprada per al prototip	85 €
Total		225 €

Els cables HDMI i USB, els caragols i els adaptadors ja estaven inclosos amb els components o es van reutilitzar, per la qual cosa no els vaig comptabilitzar com a

compres separades. El preu de la pantalla és orientatiu i es va arrodonir a partir del preu consultat per a la TS7 Pro.

8.6. Ampliacions físiques i criteri de selecció

Durant el disseny vaig considerar sensors de presència, il·luminació o temperatura ambiental, i fins i tot un microcontrolador per a tasques específiques. No els vaig afegir només perquè la placa tenia connexions disponibles: cada sensor exigeix alimentació, cablejat, programari i una explicació de la seva utilitat.

La prioritat va ser acabar una base coherent: escoltar, respondre, mostrar estat i executar accions autoritzades. Les ampliacions han de resoldre un problema concret que es pugui avaluar. Per exemple, un sensor podria ajudar a adaptar una funció a l'entorn, però això necessitaria la seva pròpia prova i no milloraria automàticament la comprensió del llenguatge.

III. AVALUACIÓ — Resultats i límits

9. Avaluació del prototip

9.1. Mesurament de l'experiència completa

La pregunta de recerca no es podia respondre únicament comprovant que ATLAS generava text. Necessitava valorar si escoltava una petició, realitzava l'acció adequada, responia i quedava preparat per a continuar utilitzant-se. Un torn que comença ràpid però executa una tasca equivocada no és un bon resultat.

Vaig separar diversos instants de mesura: final de la intervenció, enviament al model, primer fragment rebut, inici de reproducció i lliurament del resultat útil. Segons la prova, el punt de partida va ser una transcripció ja disponible o una entrada real per micròfon. Aquests temps no s'han de barrejar com si mesuressin exactament el mateix.

La latència inicial indica quan apareix la primera resposta. La latència del resultat indica quan arriba la dada o l'acció sol·licitada. Un preàmbul pot millorar la primera sense reduir la segona. També vaig observar errors de transcripció, interrupcions, reconnexions i repetició d'accions, perquè influeixen en la fiabilitat de tota la seqüència.

Les proves automatitzades ajuden a repetir casos coneguts, però tenen un abast limitat. Poden comprovar que s'ignora un esdeveniment tardà o que es retorna un error correcte; no certifiquen que una veu s'hagi sentit bé en una habitació. Per això distingeixo proves de contracte, integració, navegador i dispositiu físic.

9.2. Resultats dels mesuraments històrics

Els primers mesuraments van servir sobretot per a localitzar colls d'ampolla. En una sèrie històrica amb Realtime i delegació a un altre agent, el primer àudio d'una consulta d'emmagatzematge va tenir una mediana de 0,304 segons. No obstant això, la resposta amb el resultat útil va tenir una mediana de 16,511 segons. La reacció era ràpida; la tasca continuava sent lenta.

La major part d'aquesta espera corresponia a la delegació. El cas va ser important perquè va evitar una conclusió superficial: no podia dir que havia resolt la latència sol perquè l'agent començava a parlar en unes dècimes. Calia continuar mesurant fins a la informació que l'usuari realment havia demanat.

Una consulta d'hora va mostrar el mateix problema des del costat contrari. Una funció local podia resoldre-la de manera directa, mentre que una ruta que delegava innecessàriament va arribar a trigar més de disset segons en aquella prova. Això va motivar l'ús d'eines més directes i, posteriorment, de rutines per a resultats previsibles.

Aquestes xifres pertanyen a etapes anteriors. La xarxa, els models i les condicions no van ser idèntics en tots els assajos, per la qual cosa no les utilitzo com una comparació controlada de la versió actual. Les dades ampliades, grandàries de mostra i limitacions es conserven en l'annex B, apartats B.2 a B.4, com a evidència del procés de desenvolupament.

9.3. Proves de veu, rutines i Android

En una tanda de sis torns amb WebRTC es van registrar temps entre 1,214 i 3,075 segons des de l'entrada textual reconeguda fins a l'inici de l'àudio remot. La sessió utilitzava el model Realtime i context del projecte, però es va injectar text per a aïllar part del recorregut. Per tant, aquesta prova no mesura quant va trigar a reconèixer-se la paraula d'activació ni demostra escolta física per l'altaveu.

En una comprovació de `atlas-chat`, una consulta amb eines va produir el primer text en 2,68 segons i va completar el torn en 3,01 segons. És una evidència útil sobre aquest recorregut de text, no un temps de conversa parlada. Aquestes diferències de mètode acompanyen els resultats, en lloc de quedar amagades en una nota genèrica.

Les rutines simples van confirmar que es podia evitar treball innecessari del model. En les proves registrades, el motor local va resoldre una execució entre 8,4 i 10,9 mil·lisegons, mentre que la crida HTTP completa va prendre 53,9 mil·lisegons. A aquest recorregut cal afegir la síntesi i reproducció si l'usuari espera una resposta parlada.

En Android es van comprovar operacions no intrusives i un recorregut visual amb captures, lectura d'elements, gestos, escriptura i parada. Una mostra va registrar l'inici del control en 192,2 mil·lisegons i una captura en 471,0 mil·lisegons. Aquests nombres descriuen operacions concretes, no la durada completa de qualsevol tasca dins del telèfon.

També es va comprovar que un error recuperable de selecció no tanqués automàticament tota la sessió. Això va permetre corregir un pas sense tornar a iniciar el control des de zero. La verificació de crides o missatges reals va quedar fora

d'aquestes proves. L'annex B detalla aquesta separació entre funcionalitat disponible i evidència obtinguda.

9.4. Bateries de preguntes i millora de les accions

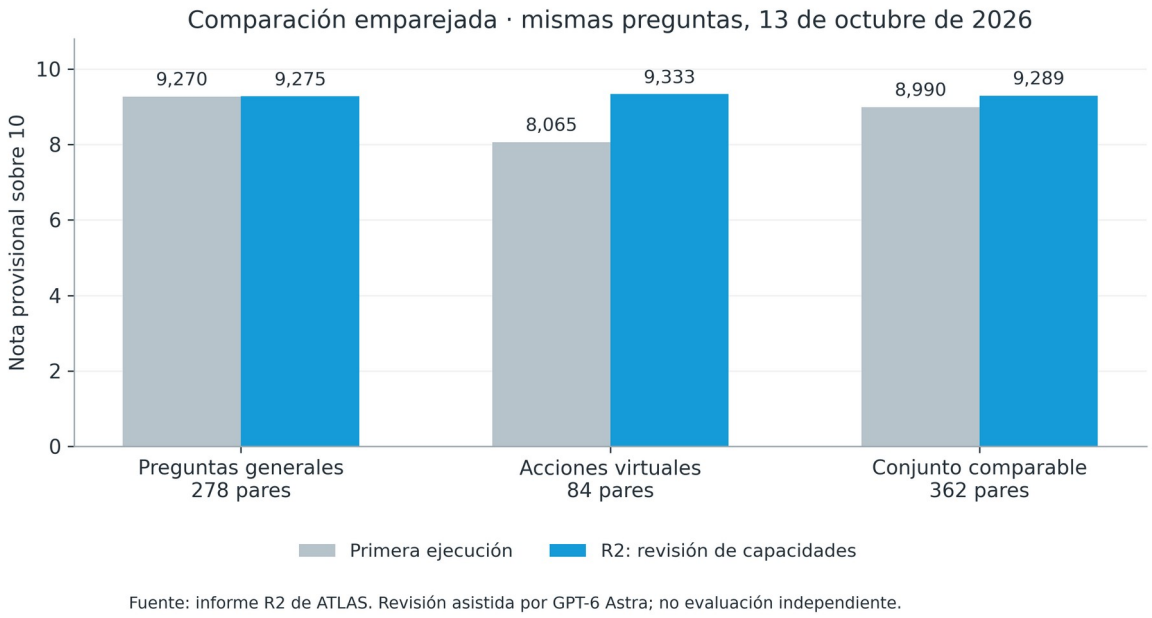
El 13 d'octubre vaig preparar 300 preguntes generals i 100 accions, com canviar el volum o reproduir música. Vaig introduir les preguntes per text, amb una sortida de veu silenciosa i accions virtuals. Vaig mesurar l'inici del motor de veu, no la primera paraula audible.

Després de revisar les fallades, vaig millorar les instruccions i els ordres i vaig repetir les mateixes 400 peticions. En els 362 casos comparables, la nota va passar de 8,990 a 9,289 sobre 10. La major millora va estar en les accions: de 8,065 a 9,333 en 84 parells. Les notes es van revisar amb ajuda de GPT-6 Astra, sense una avaluació cega, i no comproven accions físiques reals.

El 17 d'octubre vaig executar altres 400 preguntes: 100 fàcils, 100 mitjanes, 100 difícils i 100 d'interacció. Vaig incloure 70 converses de diversos torns per a comprovar referències com "i quant queda?" sense repetir el subjecte. No vaig generar veu i les accions eren virtuals.

Es van completar els 400 torns sense errors del proveïdor o de transport. La mediana fins al primer text va ser de 1,361 segons i el percentil 95, de 3,456 segons. Les comprovacions automàtiques van donar 9,60/10; aquesta puntuació indica quantes condicions programades es van complir, no és una valoració humana de les respostes.

ATLAS va respondre sobre un estat sense consultar-lo, va aplicar un volum diferent a l'anunciat i va deixar una transmissió incompleta. Vaig distingir aquestes fallades dels errors del simulador. El mètode i les comparacions estan en l'annex C, apartats C.2 i C.5 a C.7; les preguntes i respostes representatives, en l'apartat C.8.



Fuente: informe R2 de ATLAS. Revisión asistida por GPT-6 Astra; no evaluación independiente.

Figura 11. Comparació aparellada abans/després de la revisió R2. Notes de revisió assistida; les accions van ser virtuals. Elaboració pròpia a partir dels registres.

9.5. Optimització de la rapidesa i la qualitat

La millora de qualitat no va accelerar totes les respostes: en les preguntes generals, la mediana d'inici de síntesi va passar de 1,444 a 1,994 segons. Vaig preparar assajos de 20 preguntes per a separar arribada del text, preparació de la frase i arrencada de veu. Compactar les instruccions no va donar una millora fiable i va produir més preàmbuls, així que vaig descartar el canvi.

També vaig provar un ajust de l'arrencada de veu i converses amb fins a 240 torns previs. En aquests sondejos no vaig observar una degradació important per l'historial. L'ajust de veu tampoc millorava els resultats de manera consistent i, en repetir les accions, va perdre alguns encerts. Finalment vaig mantenir la versió que havia millorat la qualitat. Els assajos de latència i converses llargues es poden consultar en l'annex C, apartats C.3 i C.4.

9.6. Límits de l'avaluació

Les primeres mesures eren mostres individuals o sèries petites. Després vaig afegir les bateries de 400 preguntes, però continuen mesurant aspectes diferents: inici de síntesi, primer text o execució d'accions virtuals. Per això no vaig agrupar tots els resultats en una sola mitjana. Tampoc és el mateix una nota revisada amb ajuda d'un model que una puntuació calculada mitjançant comprovacions automàtiques.

També va haver-hi diferències entre l'entorn de proves i l'ús diari. Una sessió recentment preparada, un telèfon amb una altra aplicació en primer pla o una xarxa diferent poden modificar el resultat. El reconeixement de veu depèn a més del micròfon, la distància, el soroll i el comportament del navegador.

Aquestes campanyes mesuren la resposta en els entorns descrits; no substitueixen un mesurament del recorregut acústic complet, des del final de la veu de l'usuari fins a la primera paraula audible. L'annex C recull les proves realitzades, inclosos les seves fallades.

El nombre de proves de programari superades tampoc ha d'utilitzar-se com a sinònim de qualitat total. Demostra cobertura de casos definits, però pot deixar fos un error que ningú havia previst. Una conclusió defensable combina aquestes comprovacions amb observació real i explica l'abast dels resultats.

IV. CONCLUSIONS I CONTINUÏTAT DEL PROJECTE

10. Resposta a la pregunta inicial

El prototip va permetre comprovar que era viable unir conversa, context, eines i un dispositiu físic petit. ATLAS va deixar de ser una idea de xat i va passar a disposar de diversos canals d'interacció i d'operacions sobre el seu entorn. En aquest sentit, la hipòtesi principal va quedar sustentada pel desenvolupament realitzat.

La confirmació té límits. El sistema depèn de serveis externs per a la seva capacitat conversacional principal, necessita supervisió en tasques complexes i no totes les funcions han rebut el mateix grau de validació. Per això no concloc que hagi construït un agent infal·lible ni un producte comercial.

El canvi més important va ser substituir la cadena inicial per una estructura centrada en Realtime i eines directes. OpenClaw va aportar una base útil durant una etapa extensa, però el flux que havia construït al voltant acumulava massa espera per a l'experiència que buscava. Desenvolupar una arquitectura pròpia va permetre controlar millor el recorregut sense renunciar a l'après en el prototip anterior.

11. Aprenentatges de la integració del sistema

La principal dificultat no va ser únicament triar un bon model. Un agent útil necessita saber quines eines té, rebre resultats fiables i gestionar correctament el que ocorre entre una petició i la següent. Una fallada d'àudio o una acció incompleta poden fer que sembli poc intel·ligent encara que la interpretació inicial fos correcta.

També vaig aprendre que rapidesa i autonomia han d'avaluar-se amb cura. Parlar abans no significa acabar abans, i utilitzar més eines no significa raonar millor. Algunes tasques es beneficien d'un model flexible; unes altres es resolen millor amb una rutina petita i predictible. Triar entre ambdues va ser una part central del disseny.

El maquinari va convertir aquestes decisions en una cosa tangible. La pantalla, el micròfon, els altaveus i el sistema operatiu van fer possible utilitzar ATLAS com un dispositiu dedicat. Al mateix temps, les dependències entre alimentació, àudio i sessió gràfica van mostrar que no es podia tractar l'electrònica com una simple llista de peces connectades.

La documentació també va acabar formant part del funcionament de l'agent. Organitzar el coneixement en arxius amb responsabilitats clares va ajudar a mantenir el seu context i a explicar el projecte. No obstant això, una instrucció ben redactada no reemplaça una comprovació ni un permís aplicat per programari.

12. Desenvolupament futur

La carcassa va quedar completada com a model tridimensional amb quatre peces, ventilació, accés als ports, distribució acústica i reserves per als cables. El disseny integra les cotes de la pantalla i el micròfon, el polsador, els extensors i el cablejat. Es va documentar mitjançant renders, plànols i mesures del model.

Si continués amb ATLAS, m'agradaria ampliar els seus ports, interfícies i sensors, però també polir el que ja tinc. Una prioritat seria utilitzar un micròfon més petit i de major qualitat, juntament amb altaveus independents de la pantalla. L'àudio va ser una de les parts que més problemes em va donar: el micròfon podia tornar a captar les respostes de l'agent. Aquest retorn és eco acústic, i els rellotges independents de captura i reproducció poden dificultar la seva cancel·lació. Revisaria la posició dels components, la seva sincronització i la cancel·lació de ressò com un conjunt (IETF, RFC 7874).

També dissenyaria una carcassa més elegant i avançada, amb diversos colors i materials de major qualitat, com l'alumini. Incorporaria una refrigeració més eficaç, sense descurar el soroll del ventilador, el pes ni la recepció sense fil. Hauria d'adaptar el muntatge al nou maquinari i comprovar de nou el seu comportament tèrmic i acústic.

Una altra possibilitat seria incorporar una bateria amb un sistema com PiPower 5, per a utilitzar ATLAS sense estar sempre connectat a un endoll. Abans hauria de mesurar el consum del conjunt i comprovar el corrent disponible en mode bateria. La seva documentació recomana apagar la Pi 5 en perdre l'alimentació externa si consumeix més de 3 A, per la qual cosa no puc donar per assegurada l'autonomia de l'A1 complet. Amb una alimentació adequada i un apagat ordenat, també reduiria el risc de corrupció d'arxius durant un tall de corrent (SunFounder, PiPower 5).

En les interfícies, continuaria polint el WebScreen i l'experiència d'ús d'ATLAS. També m'agradaria millorar i optimitzar l'aplicació de control per a Android, i ampliar les capacitats d'Android Use. Revisaria la selecció d'elements, la navegació i la

comprovació dels resultats, mantenint els permisos i les confirmacions en les accions sensibles.

En la veu, buscaria una entonació i una dicció més naturals, amb canvis de ritme, pauses i èmfasis adequats a cada resposta. Voldria que ATLAS sonés més pròxim, sense exagerar les emocions.

Finalment, m'agradaria integrar més models i proveïdors d'intel·ligència artificial. L'arquitectura està molt vinculada a gpt-realtime-2.1 d'OpenAI, per la qual cosa no bastaria amb substituir el nom del model. Separaria millor la connexió amb cada proveïdor i comprovaria els torns de conversa, les eines, les imatges i la veu en cada cas.

En programari queden assajos prolongats d'estabilitat, el mesurament del recorregut acústic complet amb un protocol de latència homogeni i una revisió de les funcions sensibles d'Android sota condicions segures. També serà necessari completar l'aïllament efectiu de perfils, la gestió d'actualitzacions i una política clara de conservació i esborrat de dades.

En l'accés remot, m'agradaria dissenyar una aplicació d'autenticació basada en TOTP, amb codis temporals per a entrar en atlas.samilososami.com/try i atlas.samilososami.com/chat, sense dependre únicament d'una contrasenya estàtica (IETF, RFC 6238). Hauria de protegir el secret que genera els codis, limitar els intents i preparar una recuperació segura si perdés el telèfon. Abans d'oferir el dispositiu a altres persones, també revisaria els permisos, la recuperació física i la configuració inicial.

El resultat és un sistema funcional i una recerca sobre com construir-lo. El més valuós és mostrar les funcions integrades, l'estructura que aconsegueix una conversa més útil, les limitacions identificades i la manera de comprovar millores posteriors.

Referències documentals

Les fonts següents recolzen conceptes i especificacions. Els resultats del prototip procedeixen dels seus registres i de la memòria de desenvolupament, recopilats en els annexos B i C. La documentació inicial es va consultar l'1 d'octubre de 2026; les referències afegides després, el 18 i el 19 d'octubre. Els enllaços permeten consultar les fonts utilitzades.

ANDROID DEVELOPERS. *AccessibilityService*. Referència oficial (https://developer.android.com/reference/android/accessibilityservice/AccessibilityService). Consulta: 1 d'octubre de 2026.

ANDROID DEVELOPERS. *Foreground service types*. Documentació oficial (https://developer.android.com/develop/background-work/services/fgs/service-types). Referència tècnica del desenvolupament.

DEBIAN PROJECT. *Debian 13 "trixie"*. Informació de la distribució (https://www.debian.org/releases/trixie/index). Referència de la base del sistema.

ELEVENLABS. *Latency optimization*. Guia tècnica (https://elevenlabs.io/docs/eleven-api/guides/how-to/best-practices/latency-optimization). Referència de síntesi i transmissió d'àudio.

GONZÁLEZ KAMEL, Sami. *ATLAS: memòria inicial i registres de desenvolupament*. Material propi: document original, historial del projecte i proves recollides en els annexos. 2026.

GONZÁLEZ KAMEL, Sami. *ATLAS*. Repositori del projecte (https://github.com/samilososami/ATLAS). Codi, manuals i documentació de desenvolupament; conté components de tercers identificats per les seves respectives llicències.

MANUAL DEL CENTRE. *Dossier del Treball de Recerca*. Fotografies facilitades l'1 d'octubre de 2026. Pàgines aportades: índex, 2–3, 12, 14–25, 27–30 i 34–35. Traducció de consulta: .atlas/tdrman/MANUAL-TDR-ES.md.

NETWORKMANAGER PROJECT. *Manual pages*. Manuals de xarxa (https://networkmanager.dev/docs/man-pages/). Referència tècnica del desenvolupament.

OPENAI. *GPT Realtime 2.1*. Fitxa del model (https://developers.openai.com/api/docs/models/gpt-realtime-2.1). Consulta: 1 d'octubre de 2026.

OPENAI. *Realtime conversations*. Guia de converses (https://developers.openai.com/api/docs/guides/realtime-conversations). Referència de sessions, àudio i eines.

OPENAI. *Prompting Realtime models*. Guia d'instruccions (https://developers.openai.com/api/docs/guides/voice-prompting). Referència del disseny conversacional.

OPENAI. *Understanding and counting tokens*. Explicació del recompte de tokens (https://help.openai.com/es-es/articles/4936856-understanding-and-counting-tokens). Consulta: 1 d'octubre de 2026.

OPENCLAW. *OpenClaw Docs*. Documentació general (https://docs.openclaw.ai/). Referència de l'etapa inicial, no dependència de l'arquitectura actual.

RASPBERRY PI LTD. *Raspberry Pi 5*. Característiques oficials (https://www.raspberrypi.com/products/raspberry-pi-5/). Consulta: 1 d'octubre de 2026.

RASPBERRY PI LTD. *Configuration*. Documentació del sistema (https://www.raspberrypi.com/documentation/computers/configuration.html). Referència tècnica del muntatge.

SUNFOUNDER. *TS7-Pro: Hardware introduction*. Documentació de la pantalla (https://docs.sunfounder.com/projects/ts7-pro/en/latest/hardware_introduction.html). Consulta: 1 d'octubre de 2026.

TAILSCALE. *Connection types*. Connexions directes i retransmeses (https://tailscale.com/docs/reference/connection-types). Consulta: 1 d'octubre de 2026.

WEB AUDIO COMMUNITY GROUP. *Web Speech API*. Especificació (https://webaudio.github.io/web-speech-api/). Referència del reconeixement i síntesi en navegador.

WHISKEYSOCKETS. Baileys. Client de WhatsApp i documentació del projecte. https://github.com/WhiskeySockets/Baileys Consulta: 18 d'octubre de 2026.

BAMBU LAB. H2S: característiques i volum d'impressió. https://blog.bambulab.com/h2s-the-ultimate-single-nozzle-3d-printer-now-bigger-than-ever/ Consulta: 18 d'octubre de 2026.

BAMBU LAB. Bambu Studio. Codi i documentació del laminador. https://github.com/bambulab/BambuStudio Consulta: 18 d'octubre de 2026.

FREECAD. Projecte i documentació del modelador paramètric. https://www.freecad.org/ Consulta: 18 d'octubre de 2026.

RASPBERRY PI LTD. Active Cooler. Refrigeració de la placa. https://www.raspberrypi.com/products/active-cooler/ Consulta: 18 d'octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: bateria de 400 interaccions, 17 d'octubre de 2026. https://github.com/samilososami/ATLAS/tree/main/benchmarks/20261017-interactions Consulta: 18 d'octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: catàleg d'expressions i renderitzador. https://github.com/samilososami/ATLAS/tree/main/expressions Consulta: 18 d'octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: codi font i documentació pública del projecte. https://github.com/samilososami/ATLAS Consulta: 18 d'octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: web de presentació i portal protegit. https://atlas.samilososami.com/ Consulta: 18 d'octubre de 2026.

AMAZON (2020). Introducing the All-New Echo Family—Reimagined, Inside and Out. Consulta: 19 d'octubre de 2026.

GOOGLE (2019a). Nest Mini brings twice the bass and an upgraded Assistant. Consulta: 19 d'octubre de 2026.

GOOGLE (2019b). Get help at a glance with Google Nest Hub. Consulta: 19 d'octubre de 2026.

APPLE (2020). Apple introduces HomePod mini: A powerful smart speaker with amazing sound. Consulta: 19 d'octubre de 2026.

IETF. RFC 7874. WebRTC Audio Codec and Processing Requirements, secció 5: cancel·lació de ressò acústic. Consulta: 19 d'octubre de 2026.

SUNFOUNDER. PiPower 5: característiques de la placa. Consulta: 19 d'octubre de 2026.

SUNFOUNDER. PiPower 5: programari, alimentació i apagat segur. Consulta: 19 d'octubre de 2026.

IETF. RFC 6238. TOTP: Time-Based One-Time Password Algorithm. Consulta: 19 d'octubre de 2026.